# MTH129 Introduction to AI for Mathematical Sciences (MS)

2025-2026 Academic Year
Semester 2

SMP AI Team

Xi'an Jiaotong-Liverpool University
西交利物浦大学

Backpropagation

# Table of Contents


1 Introduction
2 Gradients
3 A simple example
4 Intuitive understanding
5 Sigmoid example

# Motivation

In this section we will develop expertise with an intuitive understanding of **backpropagation**, which is a way of computing gradients of expressions through recursive application of **chain rule**. Understanding of this process and its subtleties is critical for you to understand, and effectively develop, design and debug neural networks.

# Motivation (Continued) - Neural Networks Context

We care about this problem mainly because, in neural networks, the objective function will correspond to the loss function ($L$), and the input $\mathbf{x}$ will consist of the training data and the neural network weights (including biases).

**e.g.** $L$ could be the SVM loss function and the inputs are the training data $(x_i, y_i)$, for $i = 1, \ldots n$ and the weights $\mathbf{W}$ and $\mathbf{b}$.

$$L(x_i, y_i, \mathbf{W}, \mathbf{b}) = \max\Big(0,\ 1 - y_i\, f(x_i, \mathbf{W}, \mathbf{b})\Big)$$

where $i$ refers to the $\mathrm{i_{th}}$ data training sample and $f$ the neural network mapping.

Let's think of the training data as given and fixed, and of **the weights as variables we can adjust**. While backpropagation allows us to compute gradients with respect to any input (e.g., each $x_i$), in practice, we are primarily interested in computing **gradients with respect to the parameters** (e.g. $W$ and $b$). So that we can use it to perform a parameter update.

# Interpretation for gradients

Lets start simple and review some definition.

Consider a simple multiplication function of two numbers:

$$f(x, y) = xy$$

With basic calculus knowledge, we know:

$$\frac{\partial f}{\partial x} = y \qquad \frac{\partial f}{\partial y} = x$$

What do the derivatives tell us?

# Interpretation for gradients

Derivatives indicate the rate of change of a function with respect to that variable surrounding an infinitely small region near a particular point (assume $f$ is a univariate function):

$$\frac{\mathrm{d}f}{\mathrm{d}x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

A good way to think about the expression above is that when $h$ is very small, then the function $f$ is well-approximated by a straight line, and the derivative is its slope.

In other words, the derivative on each variable tells you the **sensitivity** of the whole expression on its value.

# Interpretation for gradients

Let's go back to our example.

- Set $x = -4$, $y = 3$.
- $f(x, y) = -12$.
- The derivative of $f$ on x:

$$\frac{\partial f}{\partial x} = y = 3.$$

- This tells us that if we increase the value of $x$ by a tiny amount, the effect on the whole expression would be to increase it (due to the positive sign), and by three times that amount.
- Similarly, since $\frac{\partial f}{\partial y} = x = -4$, we expect that increasing the value of y by some very small amount $h$ would decrease the output of $f$ (due to the negative sign), and by $4h$.

# Interpretation for gradients

Gradient $\nabla f$ is the vector of partial derivatives, so in this example, we have that

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right] = [y, x]\,.$$

For simplicity, we use terms the "gradient on $x$" instead of the phrase "the partial derivative on $x$".

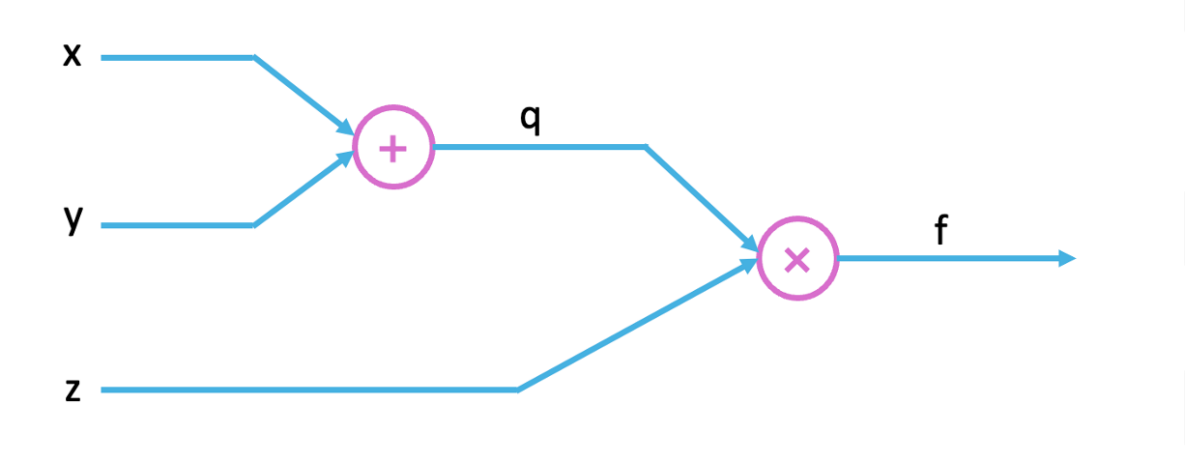
# A simple example

Let's now start to consider more complicated expressions that involve multiple composed functions, like

$$f(x, y, z) = (x + y)z$$

This expression is still simple enough to differentiate directly, but we'll take a particular approach to it that will be helpful with understanding the intuition behind backpropagation.

This function will be divided into two parts: $q = x + y$ and $f = qz$.



# A simple example

$f(x, y, z) = (x + y)z$

Set input $x = -2$, $y = 5$, $z = -4$.

**Forward pass:** computes values from inputs to output.

In forward pass, we can compute two things immediately:

- its output values
- the local gradient of its output with respect to its inputs.

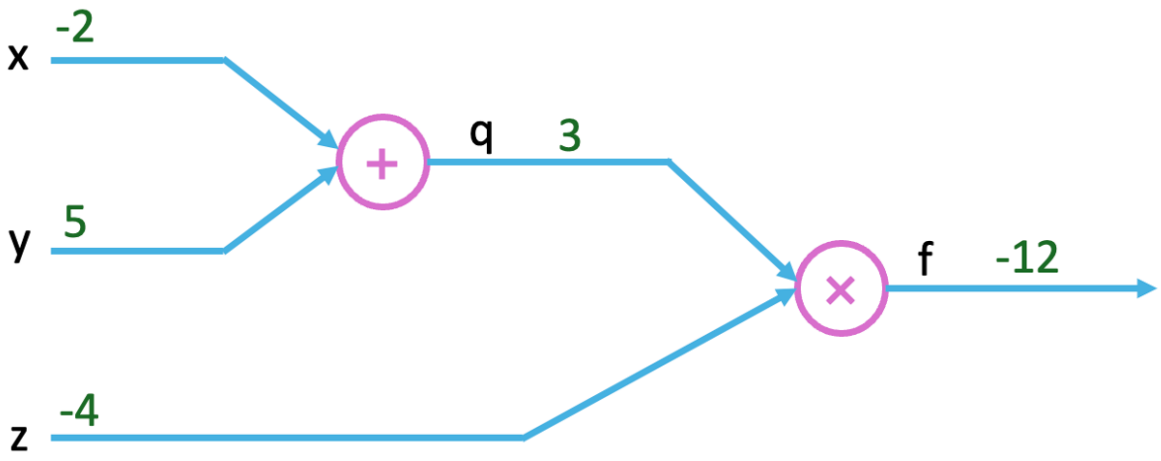
# A simple example

At the add gate, we know

- its output $q = x + y = -2 + 5 = 3$.
- local gradient $\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$

At the multiply gate, we know

- its output $f = qz = 3 \times (-4) = -12$
- local gradient $\frac{\partial f}{\partial q} = z = -4, \frac{\partial f}{\partial z} = 3$



# A simple example

We want to know $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$.

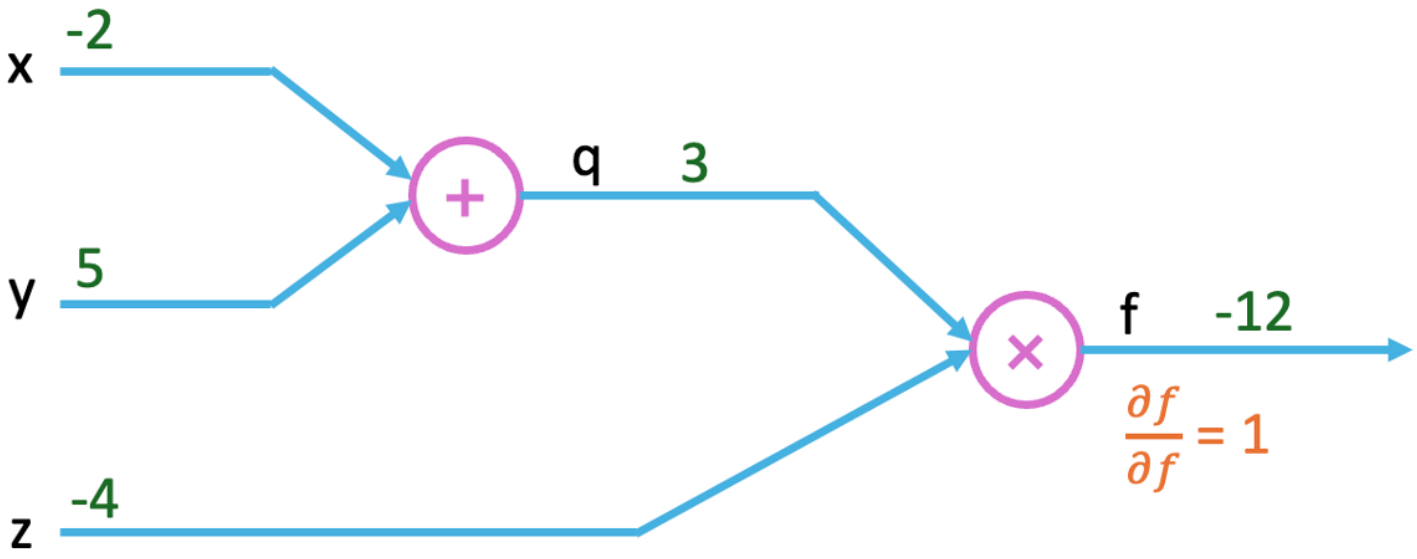
**Backward pass:** from output to inputs.

In backward pass, we learn about the gradient of each gate's output value on the final output of the entire circuit.

# A simple example

Starting from the last gate: multiply gate, we compute the gradient of the final output of this gate $f$ on the final output $f$ as

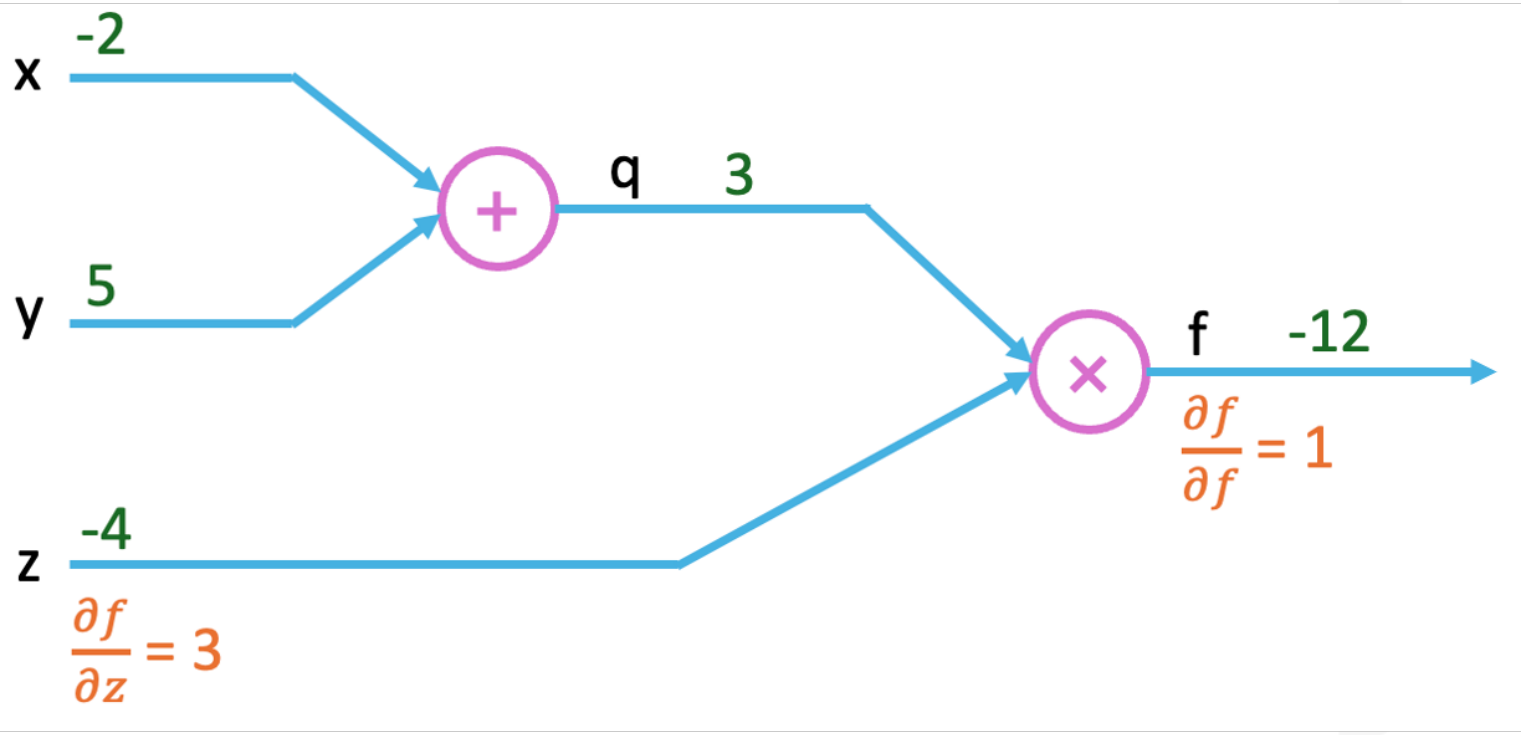
$$\frac{\partial f}{\partial f} = 1.$$



# A simple example

The gradient of $z$ on the final output $f$ can be calculated easily by

$$\frac{\partial f}{\partial z} = 3.$$

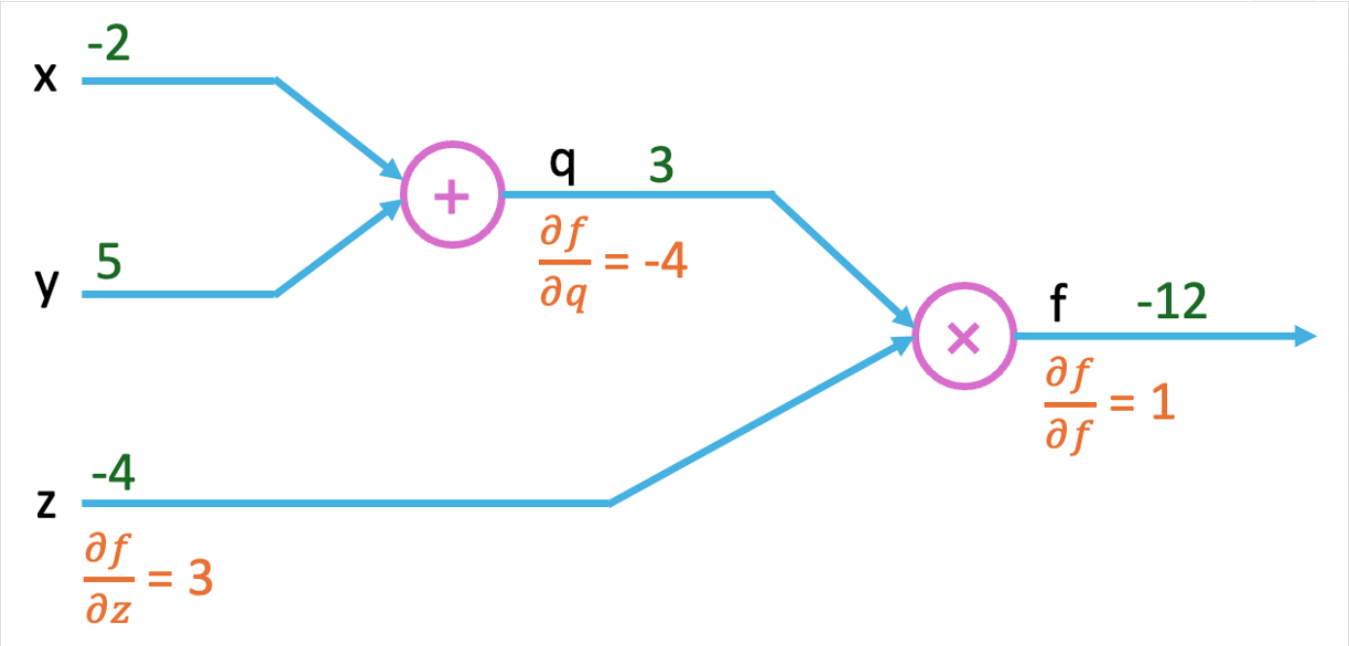


# A simple example

At the add gate, we calculate the gradient of $q$ on the final output $f$

$$\frac{\partial f}{\partial q} = -4.$$

The add gate (which is an input to the multiply gate) learns that the gradient for its output was -4. If we anthropomorphize the circuit as wanting to the final output a higher value, then we can think of the circuit as "wanting" the output of the add gate to be lower (due to negative sign), and with a force of 4.

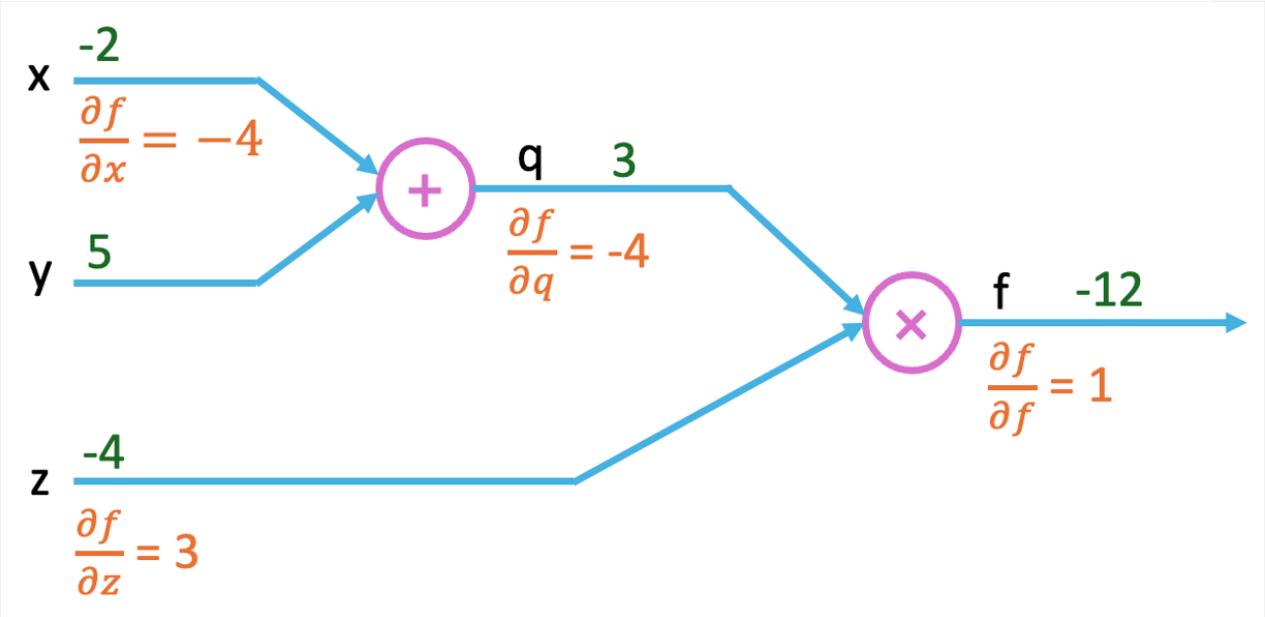


# A simple example

To continue the recurrence and to chain the gradient, the add gate takes that gradient ($\frac{\partial f}{\partial q}$) and multiplies it to the local gradient for its input $x$ ($\frac{\partial f}{\partial x}$):

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \times \frac{\partial q}{\partial x} = -4 \times 1 = -4.$$

It tells us if $x$ was to decrease (responding to their negative gradient) then the add gate's output would decrease, which in turn makes the multiply gate's output increase.

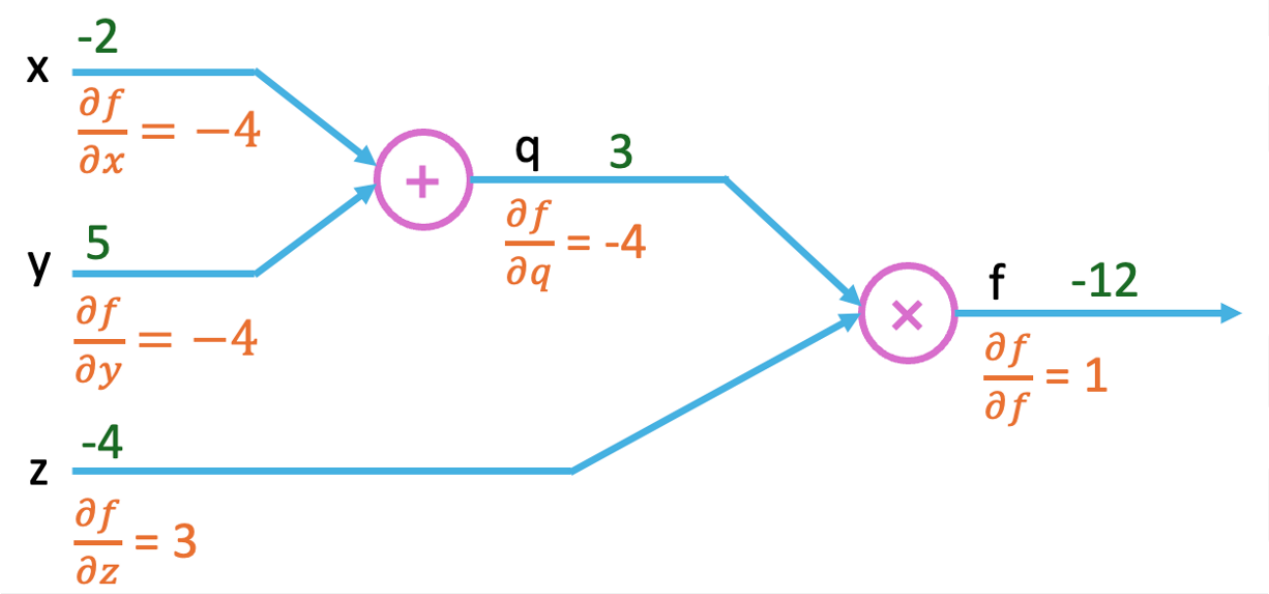


# A simple example

Similarly, we know $\frac{\partial f}{\partial y}$, according to the chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \times \frac{\partial q}{\partial y} = -4 \times 1 = -4$$

It tells us if $y$ was to decrease (responding to their negative gradient) then the add gate's output would decrease, which in turn makes the multiply gate's output increase.



# A simple example

Backpropagation can thus be thought of as gates communicating to each other (through the gradient signal) whether they want their outputs to increase or decrease (and how strongly), so as to make the final output value higher.

# Intuitive understanding of backpropagation

Notice that backpropagation is a beautifully local process.

In **forward pass**, every gate in the circuit diagram gets some inputs and can right away compute two things:

- its output value
- the local gradient of its output with respect to its inputs

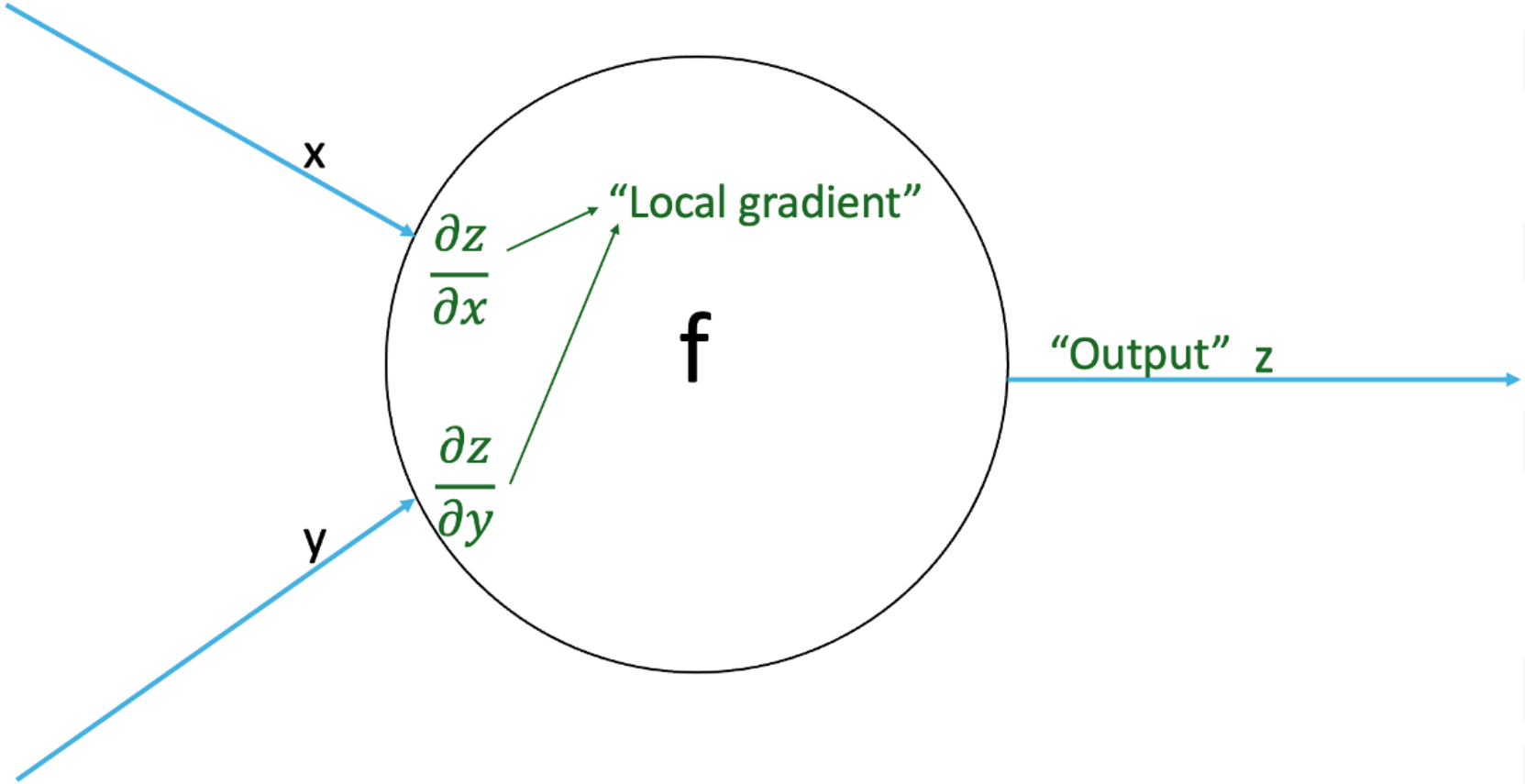
The gates can do this completely independently without being aware of any of the details of the full circuit that they are embedded in.

Once the forward pass is over, during **backward pass**, the gate will eventually learn about the gradient of its output value on the final output of the entire circuit.

Chain rule says that the gate should take that gradient and multiply it into every gradient it normally computes for all of its inputs.

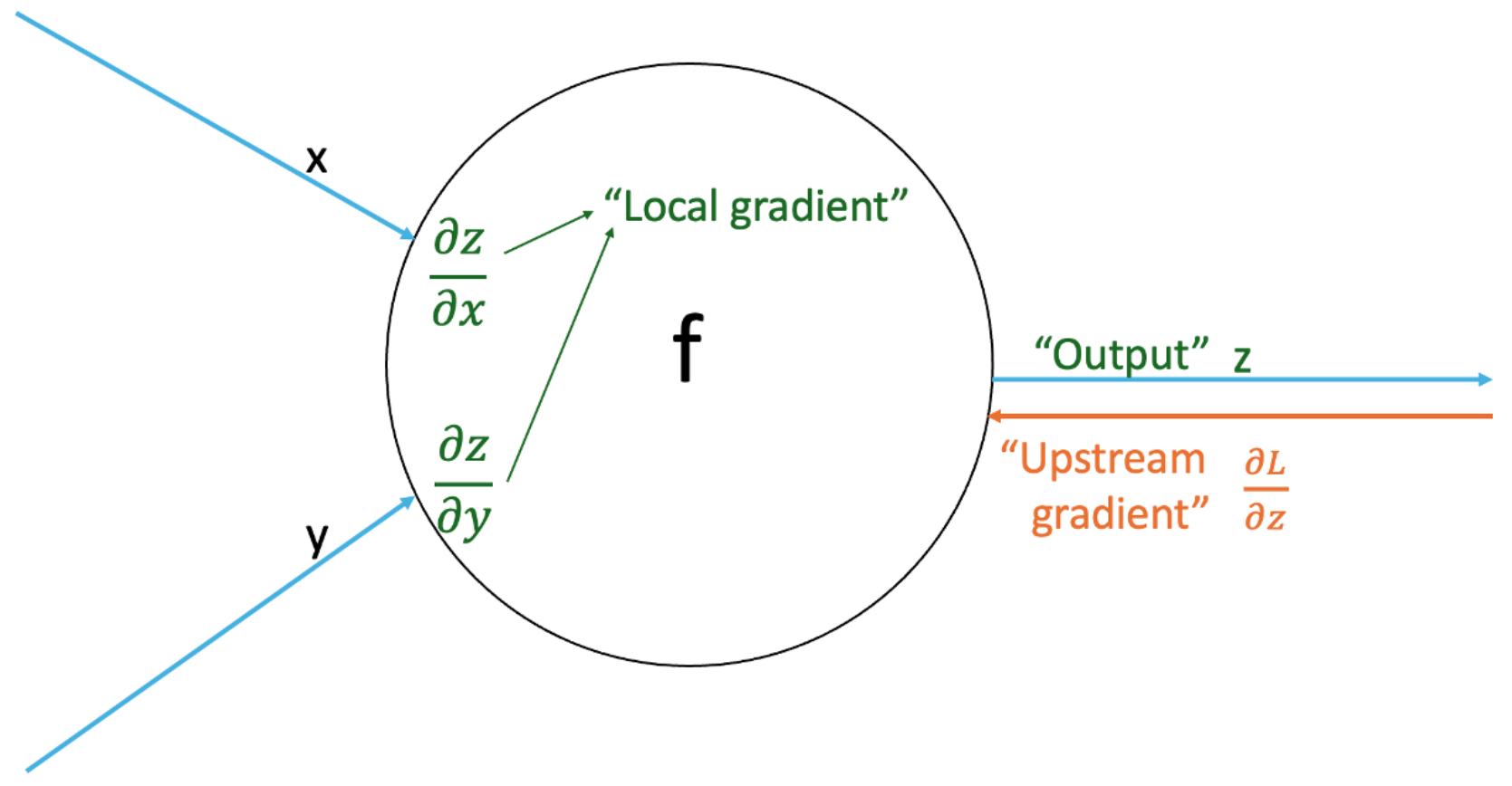
# Intuitive understanding of backpropagation

Forward pass: calculate output $z$, local gradient $\frac{\partial z}{\partial x}$, and $\frac{\partial z}{\partial y}$.



# Intuitive understanding of backpropagation

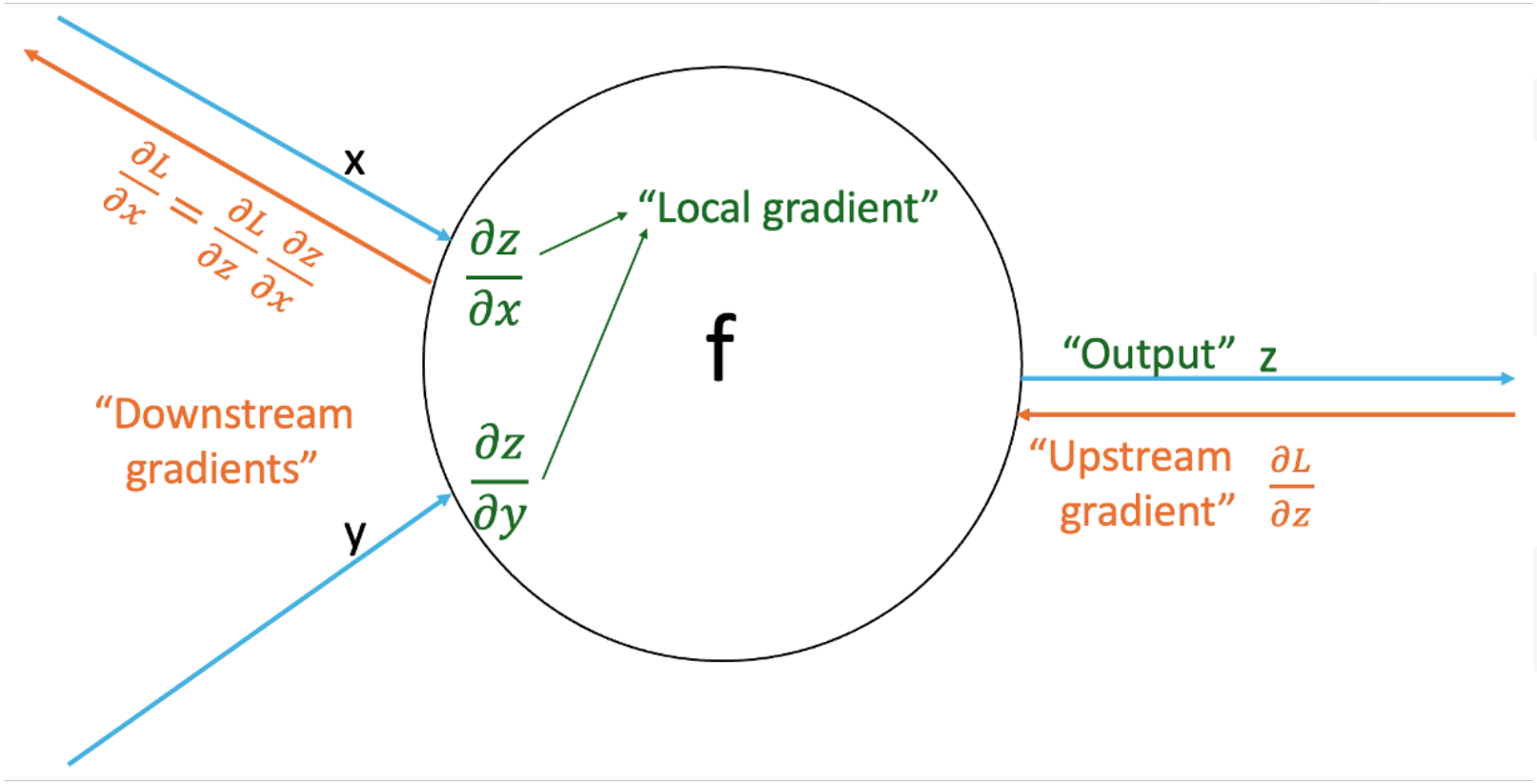
Backward pass: learn upstream gradient $\frac{\partial L}{\partial z}$.



# Intuitive understanding of backpropagation

Backward pass: compute the downstream gradient by multiplying the upstream gradient with the local gradient.

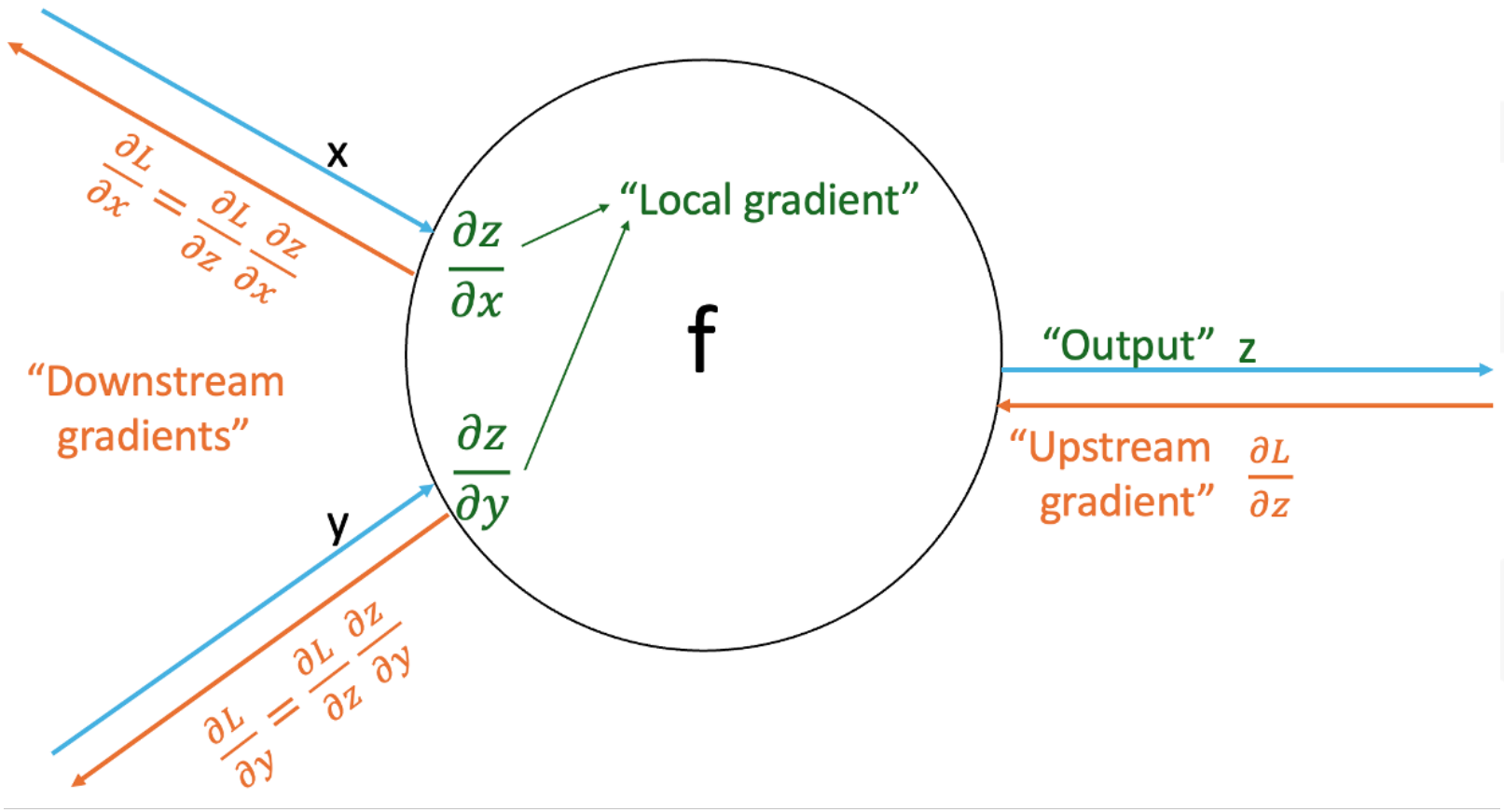
$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}.$$



# Intuitive understanding of backpropagation

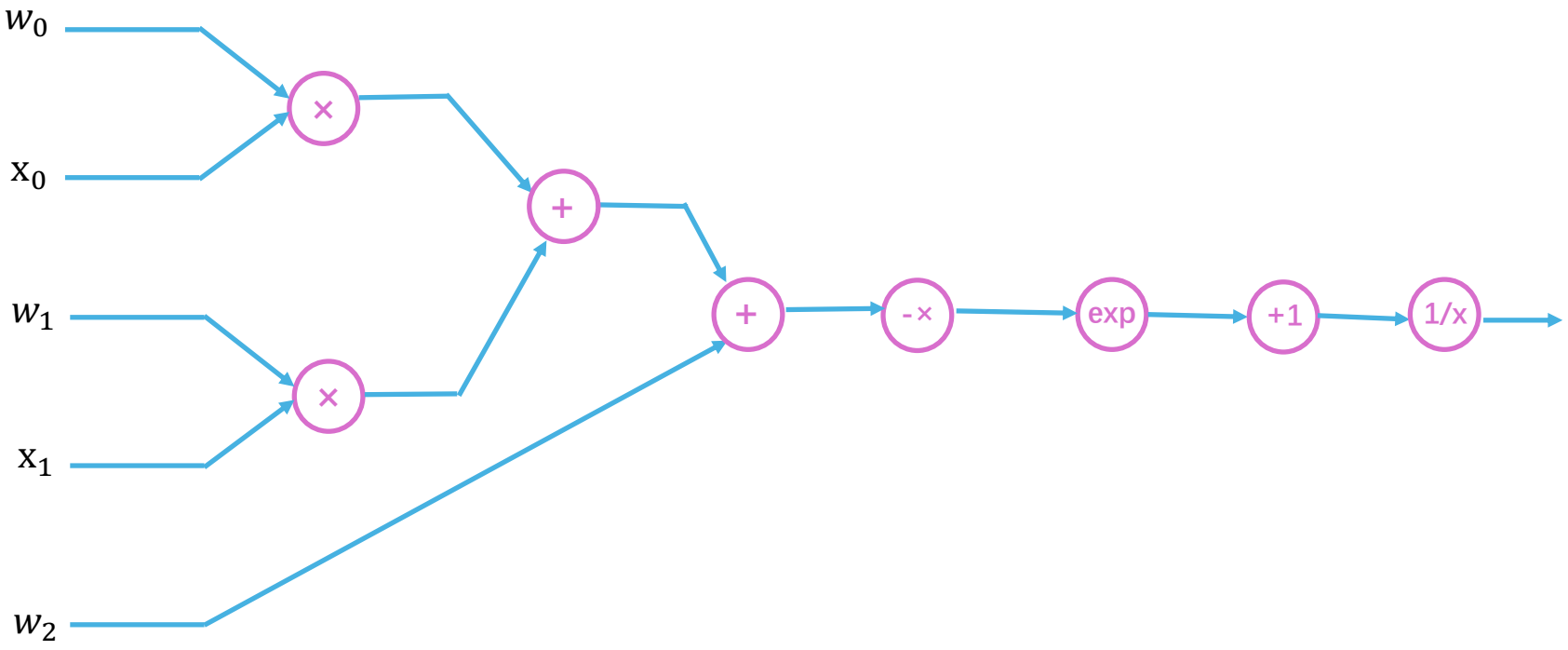
Backward pass: compute the downstream gradient by multiplying the upstream gradient with the local gradient.

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}.$$



# Modularity: Sigmoid example

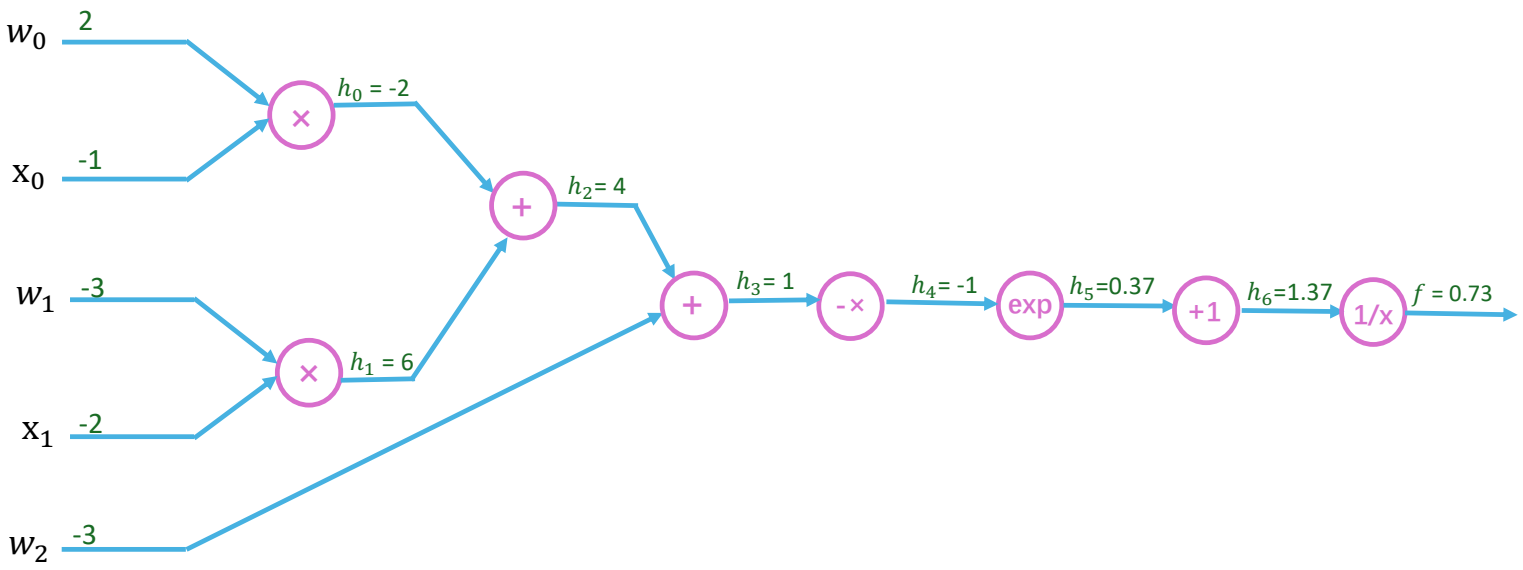
$$f(w, x) = \frac{1}{1 + \exp(-(w_1 x_1 + w_2 x_2 + w_0))}$$



# Modularity: Sigmoid example

- $f(w, x) = \frac{1}{1+\exp(-(w_0 x_0 + w_1 x_1 + w_2))}$
- Set $w_0 = 2$, $x_0 = -1$, $w_1 = -3$, $x_1 = -2$, $w_1 = -2$, $w_2 = -3$.

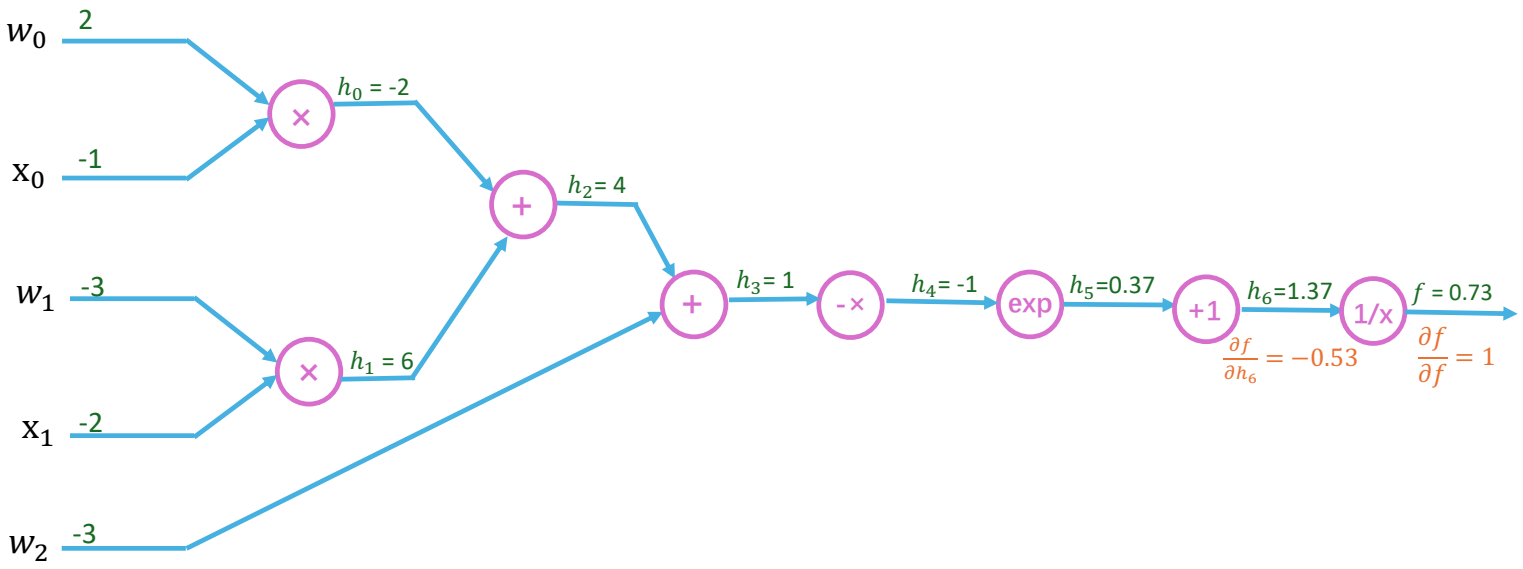
Forward pass: calculate each gate's output and local gradient



# Modularity: Sigmoid example

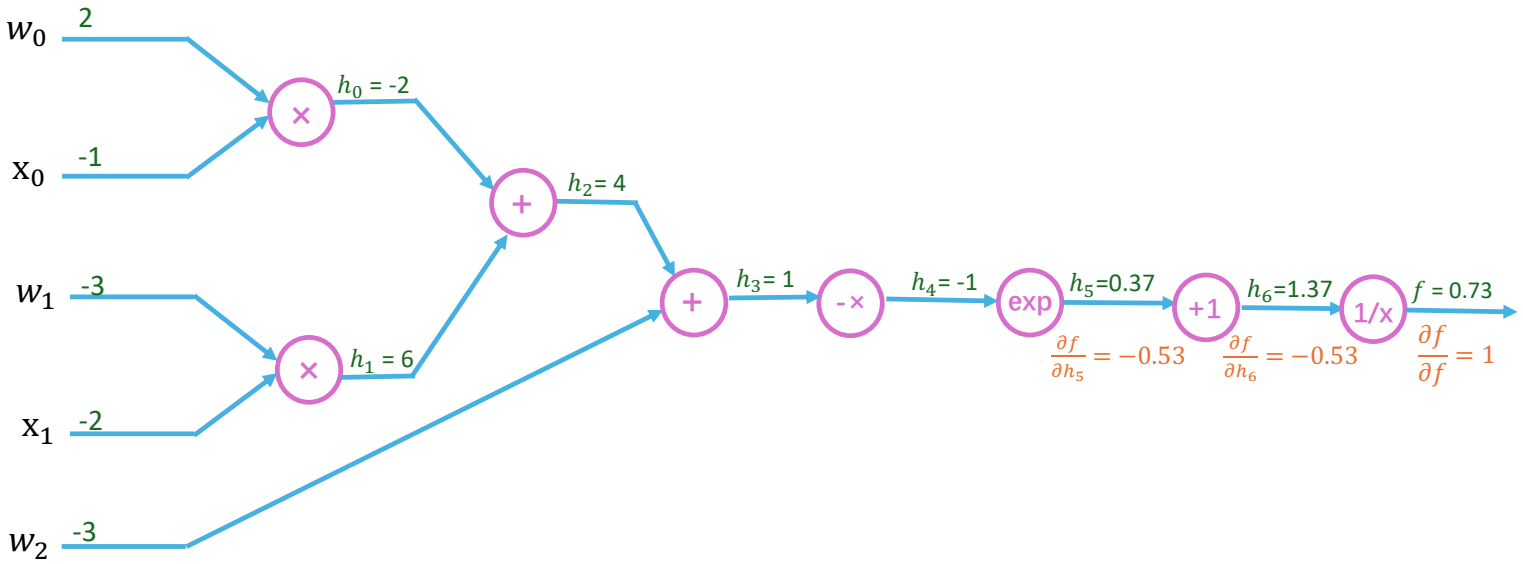
Backward pass: calculate the gradient of output in each gate on the final output $f$.

- $\frac{\partial f}{\partial f} = 1$
- $\frac{\partial f}{\partial h_6} = \frac{\partial f}{\partial f}\frac{\partial f}{\partial h_6} = 1 \times \left(-\frac{1}{x^2}\right) = 1 \times \left(-\frac{1}{1.37^2}\right) = -0.53$.(Multiply the upstream gradient $\frac{\partial f}{\partial f}$ and the local gradient i.e. the gradient of each gate's input on its output $\frac{\partial f}{\partial h_6}$.)



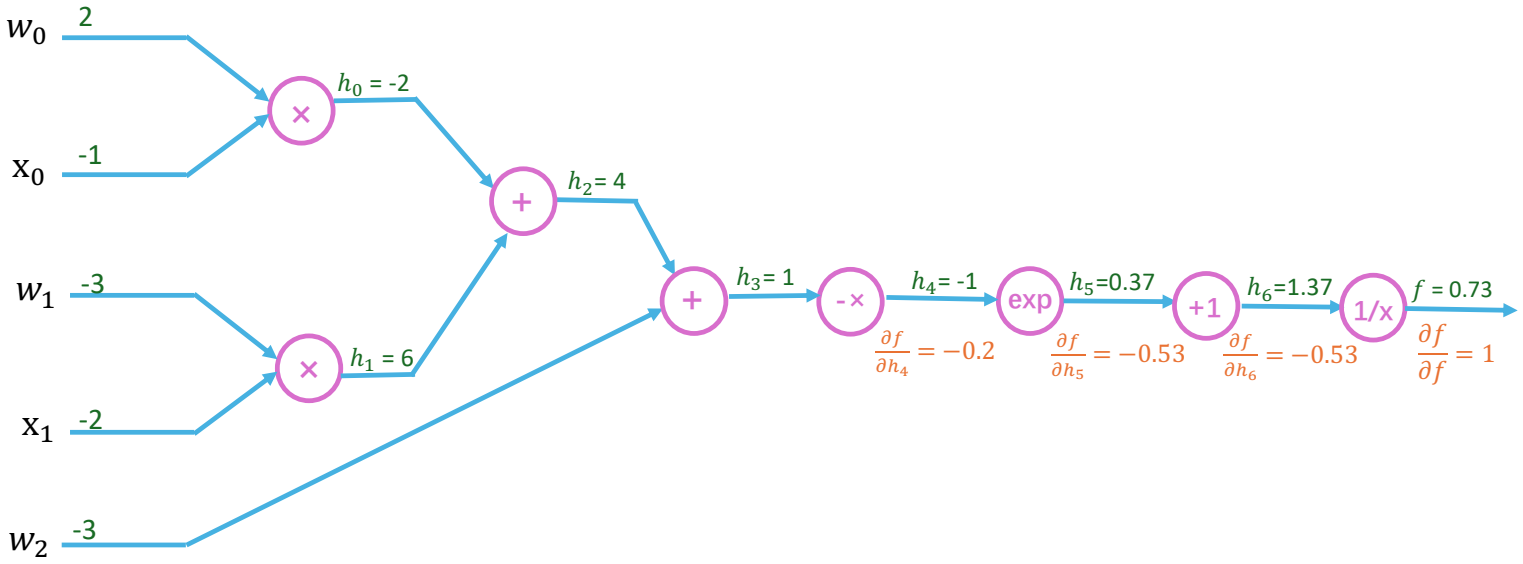
# Modularity: Sigmoid example

Because $h_6 = h_5 + 1$, from the forward pass, we know $\frac{\partial h_6}{\partial h_5} = 1$. Then, $\frac{\partial f}{\partial h_5} = \frac{\partial f}{\partial h_6}\frac{\partial h_6}{\partial h_5} = -0.53 \times 1 = -0.53$. (Multiply the upstream gradient $\frac{\partial f}{\partial h_6}$ and the local gradient i.e. the gradient of each gate's input on its output $\frac{\partial h_6}{\partial h_5}$.)



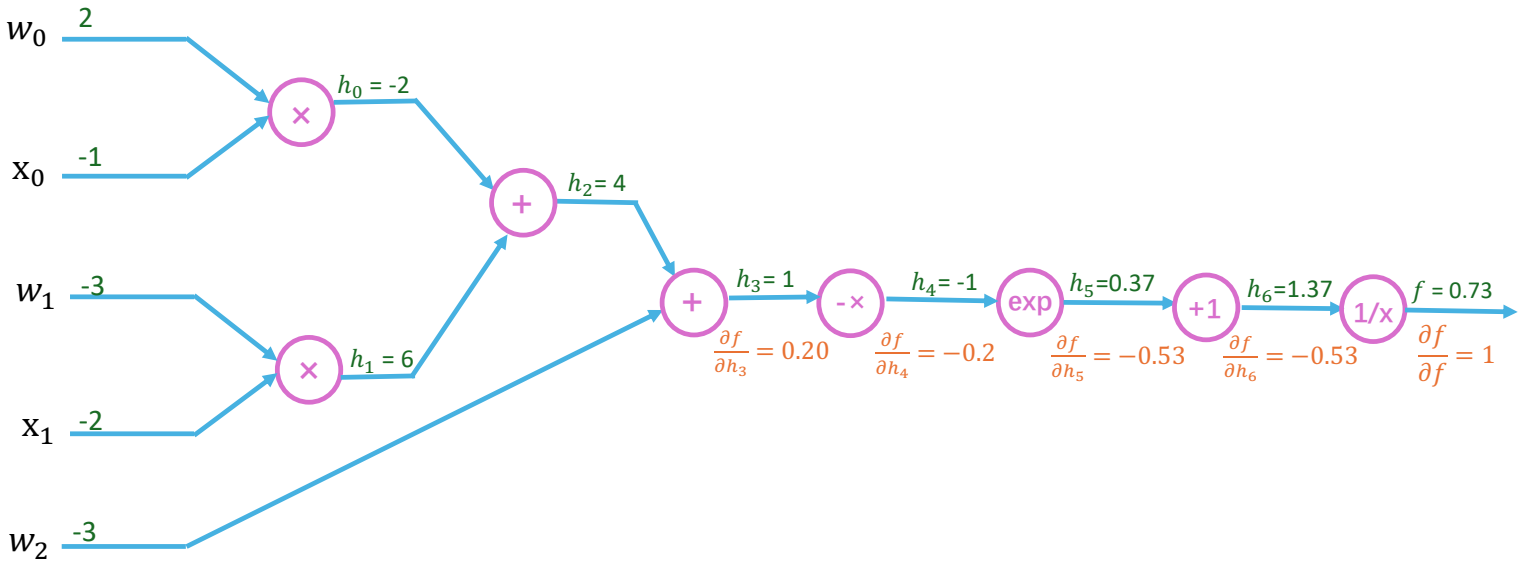
# Modularity: Sigmoid example

Because $h_5 = \exp(h_4)$, from the forward pass, we know $\frac{\partial h_5}{\partial h_4} = \exp(h_4) = 0.37$. Then, $\frac{\partial f}{\partial h_4} = \frac{\partial f}{\partial h_5}\frac{\partial h_5}{\partial h_4} = -0.53 \times 0.37 = -0.20$. (Multiply the upstream gradient $\frac{\partial f}{\partial h_5}$ and the local gradient i.e. the gradient of each gate's input on its output $\frac{\partial h_5}{\partial h_4}$.)



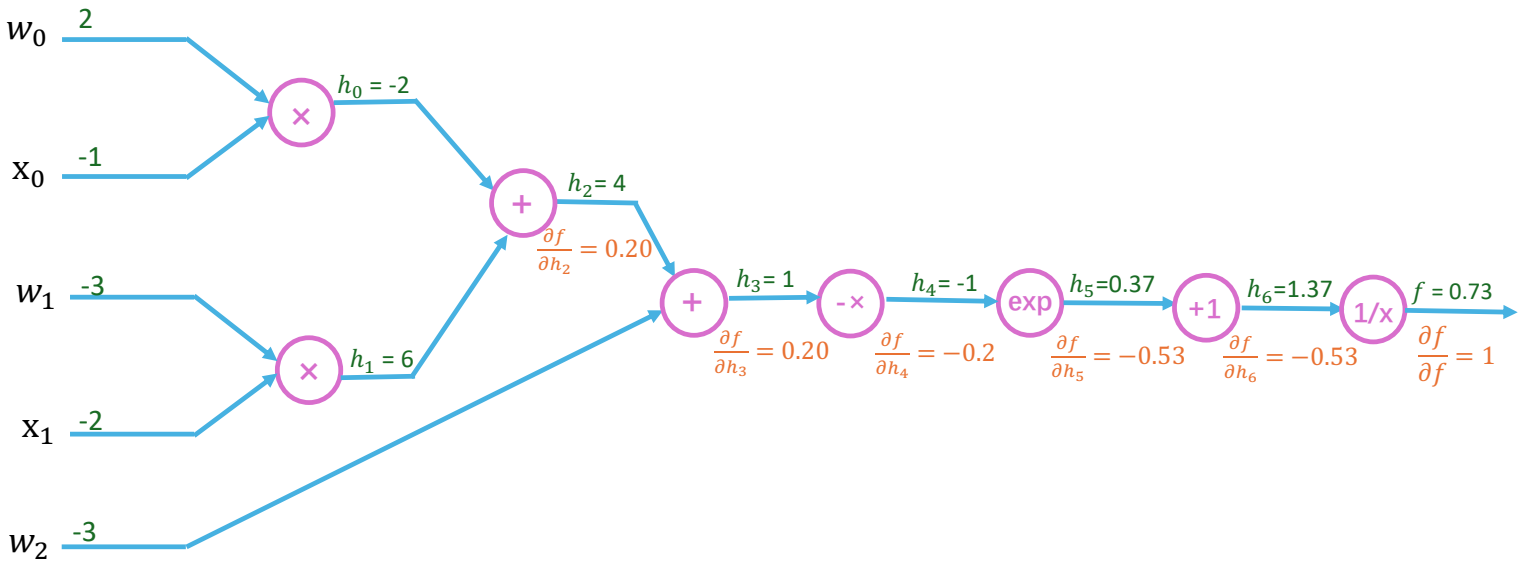
# Modularity: Sigmoid example

Because $h_4 = -h_3$, from the forward pass, we know $\frac{\partial h_4}{\partial h_3} = -1$. Then, $\frac{\partial f}{\partial h_3} = \frac{\partial f}{\partial h_4}\frac{\partial h_4}{\partial h_3} = -0.20 \times -1 = 0.20$. (Multiply the upstream gradient $\frac{\partial f}{\partial h_4}$ and the local gradient i.e. the gradient of each gate's input on its output $\frac{\partial h_4}{\partial h_3}$.)



# Modularity: Sigmoid example

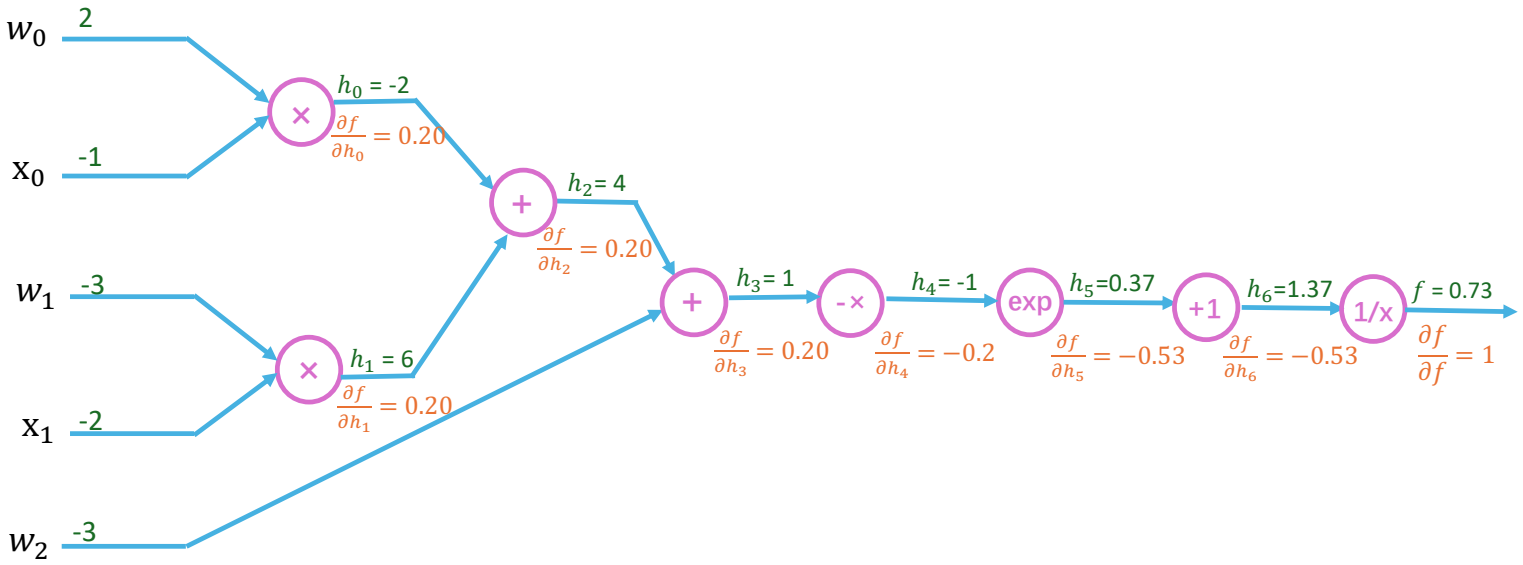
Because $h_3 = h_2 + 1$, from the forward pass, we know $\frac{\partial h_3}{\partial h_2} = 1$. Then, $\frac{\partial f}{\partial h_2} = \frac{\partial f}{\partial h_3}\frac{\partial h_3}{\partial h_2} = 0.20 \times 1 = 0.20$. (Multiply the upstream gradient $\frac{\partial f}{\partial h_3}$ and the local gradient i.e. the gradient of each gate's input on its output $\frac{\partial h_3}{\partial h_2}$.)



# Modularity: Sigmoid example

Because $h_2 = h_0 + h_1$, from the forward pass, we know $\frac{\partial h_2}{\partial h_0} = 1$ and $\frac{\partial h_2}{\partial h_1} = 1$.

- $\frac{\partial f}{\partial h_0} = \frac{\partial f}{\partial h_2}\frac{\partial h_2}{\partial h_0} = 0.20 \times 1 = 0.20$
- $\frac{\partial f}{\partial h_1} = \frac{\partial f}{\partial h_2}\frac{\partial h_2}{\partial h_1} = 0.20 \times 1 = 0.20$

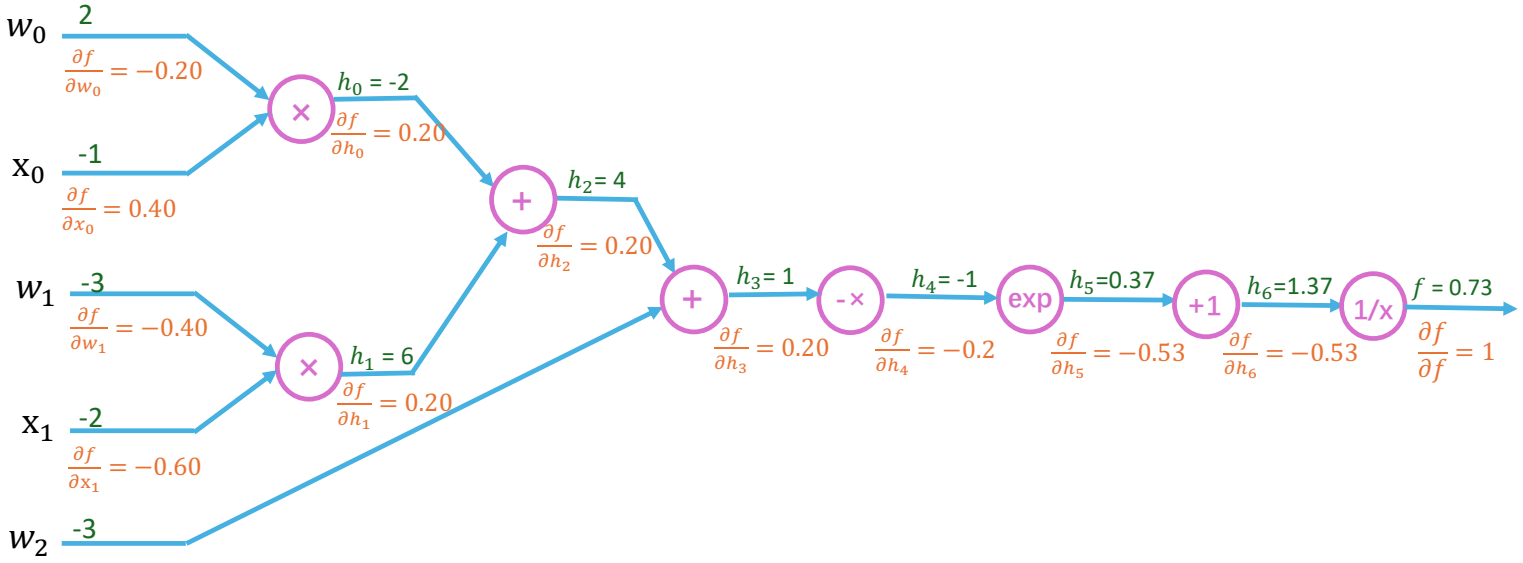


# Modularity: Sigmoid example

Because $h_0 = w_0 x_0$, from the forward pass, we know $\frac{\partial h_0}{\partial w_0} = x_0$ and $\frac{\partial h_0}{\partial x_0} = w_0$.

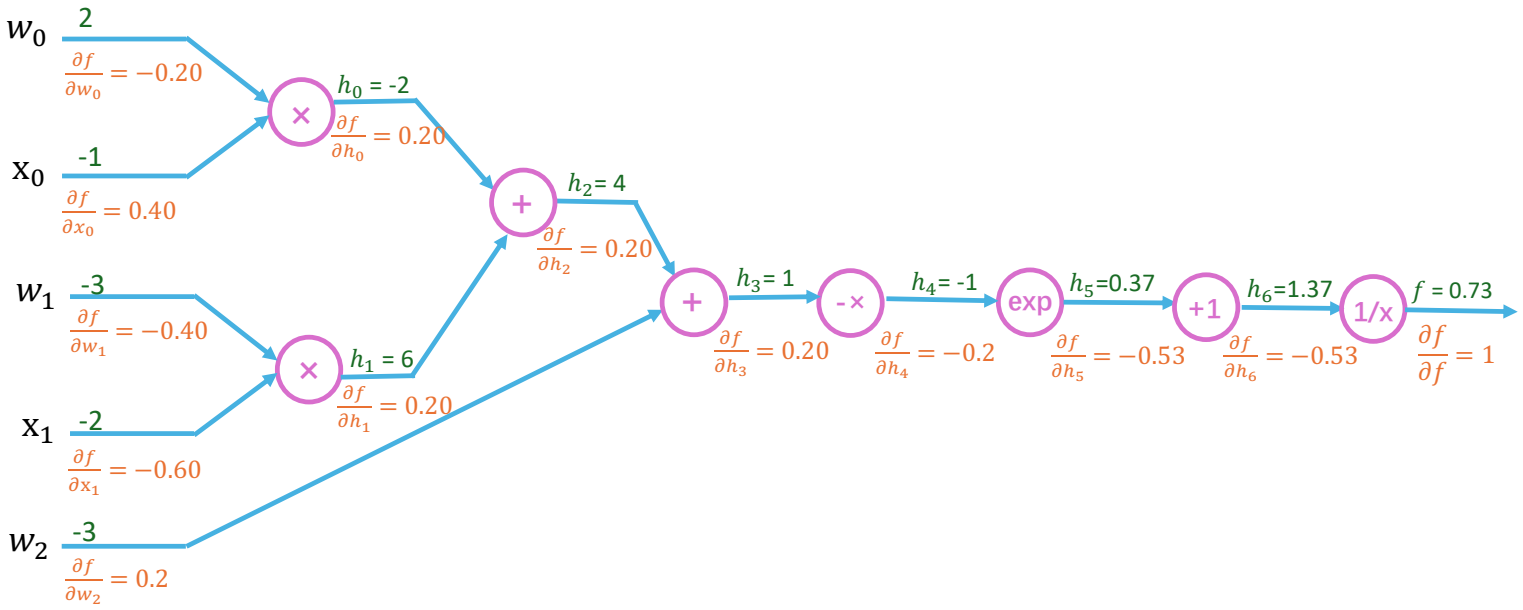
- $\frac{\partial f}{\partial w_0} = \frac{\partial f}{\partial h_0}\frac{\partial h_0}{\partial w_0} = 0.20 \times (-1) = -0.20$
- $\frac{\partial f}{\partial x_0} = \frac{\partial f}{\partial h_0}\frac{\partial h_0}{\partial x_0} = 0.20 \times 2 = 0.40$

Similarly, we can calculate $\frac{\partial f}{\partial w_1}$ and $\frac{\partial f}{\partial x_1}$.



# Modularity: Sigmoid example

Because $h_3 = h_2 + w_2$, from the forward pass, we know $\frac{\partial h_3}{\partial w_2} = 1$. Then, we have $\frac{\partial f}{\partial w_2} = \frac{\partial f}{\partial h_3}\frac{\partial h_3}{\partial w_2} = 0.20 \times 1 = 0.20$



# Modularity: Sigmoid example

In the example above, we see a long chain of function applications that operates on the result of the dot product between $w$ and $x$. The function that these operations implement is called the sigmoid function $\sigma(x)$.The derivative of the sigmoid function with respect to its input can be simplified by

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\frac{\mathrm{d}\sigma(x)}{\mathrm{d}x} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\sigma(x)$$

Through this expression, the gradient calculation will be simplified. For example, the sigmoid example receives the input 1.0, then we can calculate the output 0.73 through the forward pass. The expression also gives us the **local gradient** equals to $(1-0.73)0.73 \approx 0.2$, as shown in the detailed calculation before.
Hence, it would be very practical to group these operations into a single gate for actual use.

# Procedures of backpropagation

Now with the knowledge of forward pass and backward pass, we are ready to look into backpropagation. Let's try to break it down in a few simple steps.

**Step 1: Forward pass**

As we said before, through forward pass, we calculate

- each gate's output
- local gradient

**Step 2: Calculation of Error**

After obtaining the output, we compare it with the desired output and calculate the error. The error measures how far off the network's prediction is from the true data. It quantifies the discrepancy between the predicted and expected outputs using a loss function, such as mean squared error (MSE) or cross-entropy.

# Procedures of backpropagation

**Step 3: Backward pass**

Backward pass is where the actual learning occurs. It involves propagating the error backward through the network to update parameters (weights and biases). This process allows the network to adjust its parameters based on the calculated error. In backward pass, we compute the gradient of the loss function with respect to the parameters using the Chain Rule. The gradient indicates the direction and magnitude of the steepest ascent in the loss function's landscape. By moving in the opposite direction of the gradient, we can update the parameters to reduce the error and improve the network's performance.

# Procedures of backpropagation

**Step 4: Update parameters**

Using the gradients calculated during backward pass, we update the parameters of the network. This is typically done using an optimization algorithm like SGD or its variants. The algorithm adjusts the parameters to minimize the loss function, gradually improving the network's ability to make accurate predictions.

$$w_{new} \leftarrow w_{old} - \eta \frac{\partial L}{\partial w}$$

- $\eta$: learning rate
- $\frac{\partial L}{\partial w}$: gradient of the loss w.r.t. parameters

# Procedures of backpropagation

**Step 4: Update parameters**

Using the gradients calculated during backward pass, we update the parameters of the network. This is typically done using an optimization algorithm like SGD or its variants. The algorithm adjusts the parameters to minimize the loss function, gradually improving the network's ability to make accurate predictions.

**Summary:**

The process of forward pass, error calculation, backward pass, and parameter updates is repeated iteratively for a given number of epochs or until the network reaches a desired level of performance.

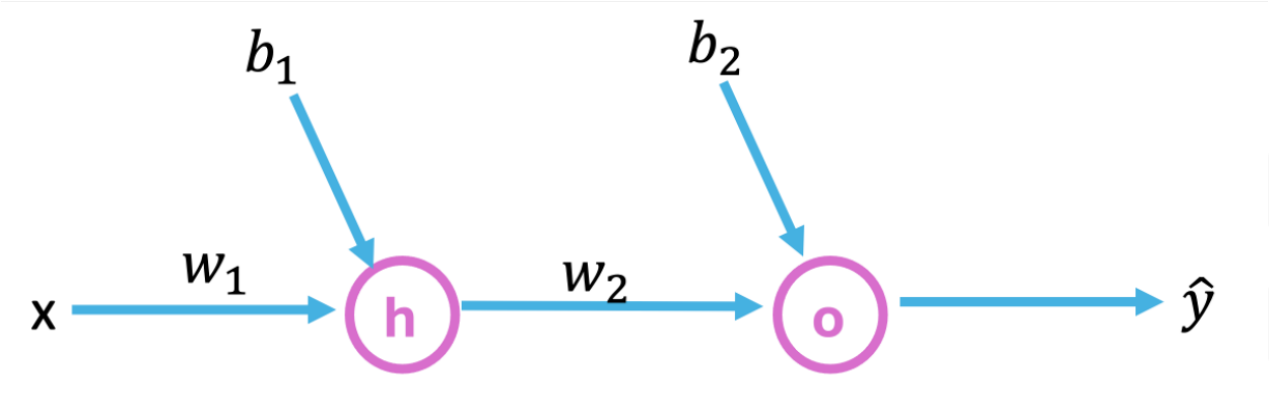
By repeatedly adjusting the parameters based on the gradients calculated during backpropagation, the neural network learns from the data, captures patterns, and improves its predictions over time.

# Example 1

Assume

- Input $x = 0.5$, true outcome $y = 1.5$
- Activation functions in hidden layer $h$ and output layer $o$ are both sigmoind function.
- $w_1 = 0.3$, $w_2 = 0.2$, $b_1 = 0.1$, $b_2 = 0.4$.
- Loss function: $L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$.
- Learning rate $\alpha = 0.1$.

We would like to adjust $w_1$, $w_2$, $b_1$, and $b_2$ to make predict value $\hat{y}$ as close as possible to the actual value $y$.



# Example 1

Step 1: Forward pass

$$h(input) = w_1 x + b_1 = 0.3 \times 0.5 + 0.1 = 0.25$$

$$h(output) = \sigma(h(input)) = \frac{1}{1 + \exp(-0.25)} = 0.5621$$

$$o(input) = w_2 h(output) + b_2 = 0.2 \times 0.5621 + 0.4 = 0.5124$$

$$\hat{y} = o(output) = \sigma(o(input)) = 0.6253$$

Step 2: Calculate error

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(1.5 - 0.6253)^2 = 0.3826$$

# Example 1

Step 3: Backward pass
We first calculate the gradient of $o(output)$ on loss $L$.

$$\frac{\partial L}{\partial o(output)} = -(y - \hat{y}) = -(1.5 - 0.6253) = -0.8747$$

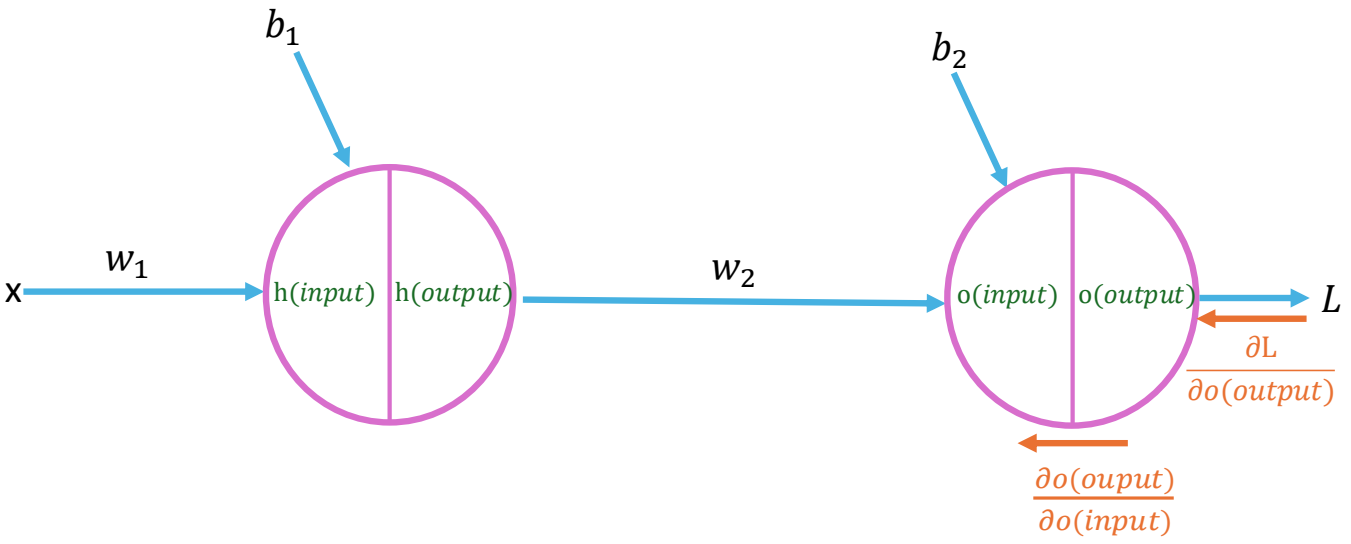
According to $o(output) = \sigma(o(input))$, the local gradient

$$\frac{\partial o(output)}{\partial o(input)} = o(output)(1 - o(output)) = 0.2343.$$

Then, gradient of $o(input)$ on the total loss are:

$$\frac{\partial L}{\partial o(input)} = \frac{\partial L}{\partial o(output)} \frac{\partial o(output)}{\partial o(input)} = -0.8747 \times 0.2343 = -0.2049.$$

# Example 1




$$\frac{\partial L}{\partial o(\text{input})} = \frac{\partial L}{\partial o(output)} \frac{\partial o(output)}{\partial o(input)}$$

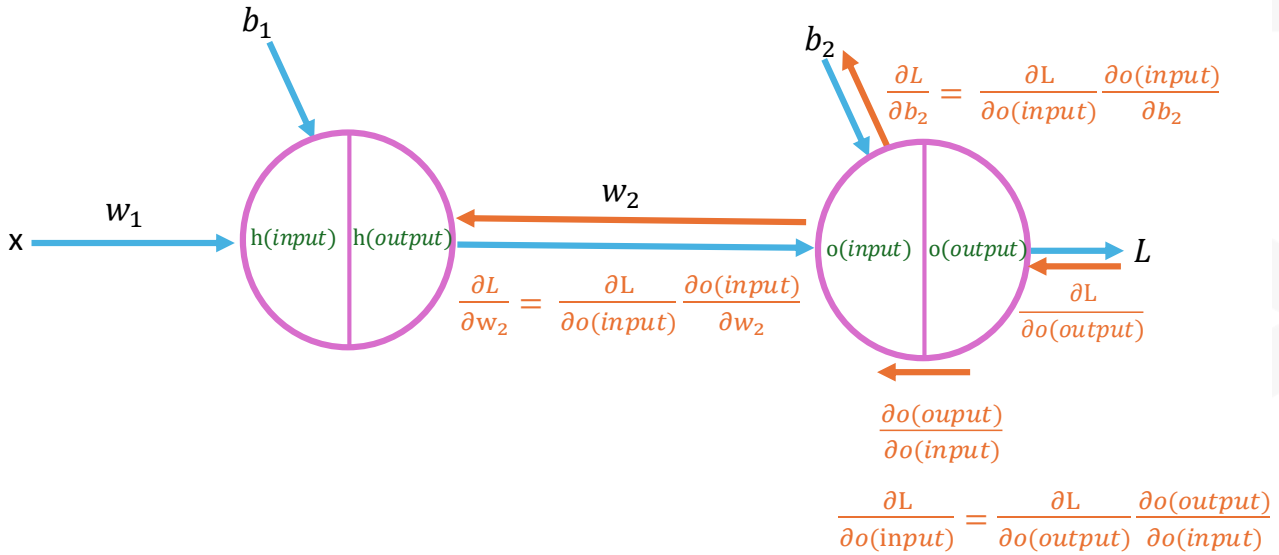
# Example 1

Because $o(input) = w_2 h(output) + b_2$, we know $\frac{\partial o(output)}{\partial w_2} = h(output) = 0.5621$ and $\frac{\partial o(output)}{\partial b_2} = 1$. Then, the gradient of $w_2$ and $b_2$ on the total loss are:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial o(input)} \frac{\partial o(input)}{\partial w_2} = -0.2049 \times 0.5621 = -0.1151$$

and

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial o(input)} \frac{\partial o(input)}{\partial w_2} = -0.2049 \times 1 = -0.2049$$



# Example 1

Similarly, $\frac{\partial o(output)}{\partial h(input)} = w_2 = 0.2$

$$\frac{\partial L}{\partial h(output)} = \frac{\partial L}{\partial o(input)} \frac{\partial o(input)}{\partial h(output)} = -0.2049 \times 0.2 = -0.0410$$

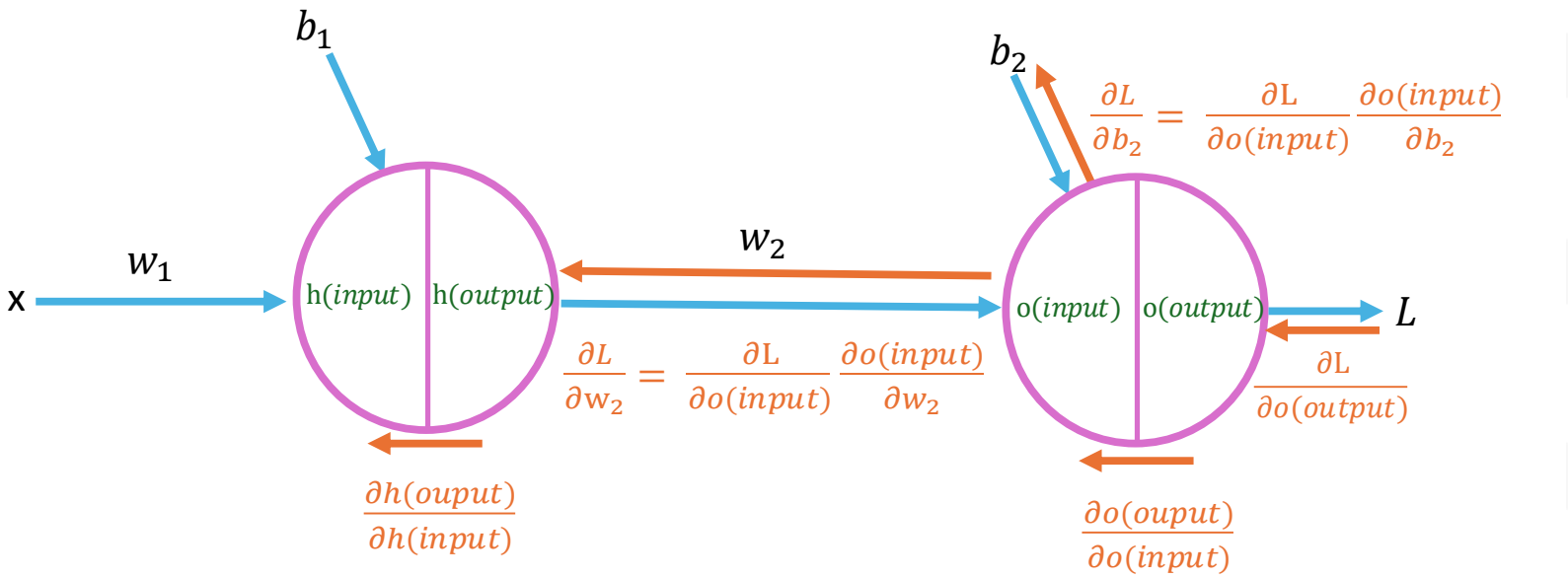
Then, the local gradient of $h$ can be calculated by

$$\frac{\partial h(output)}{\partial h(input)} = \sigma(h(output))(1 - \sigma(h(output))) = 0.2461$$

Therefore,

$$\frac{\partial L}{\partial h(input)} = \frac{\partial L}{\partial h(output)} \frac{\partial h(output)}{\partial h(input)} = -0.0410 \times 0.2461 = -0.0100$$

# Example 1




$$\frac{\partial L}{\partial h(input)} = \frac{\partial L}{\partial o(input)} \frac{\partial o(input)}{\partial h(output)} \frac{\partial h(output)}{\partial h(input)} \qquad \frac{\partial L}{\partial o(input)} = \frac{\partial L}{\partial o(output)} \frac{\partial o(output)}{\partial o(input)}$$

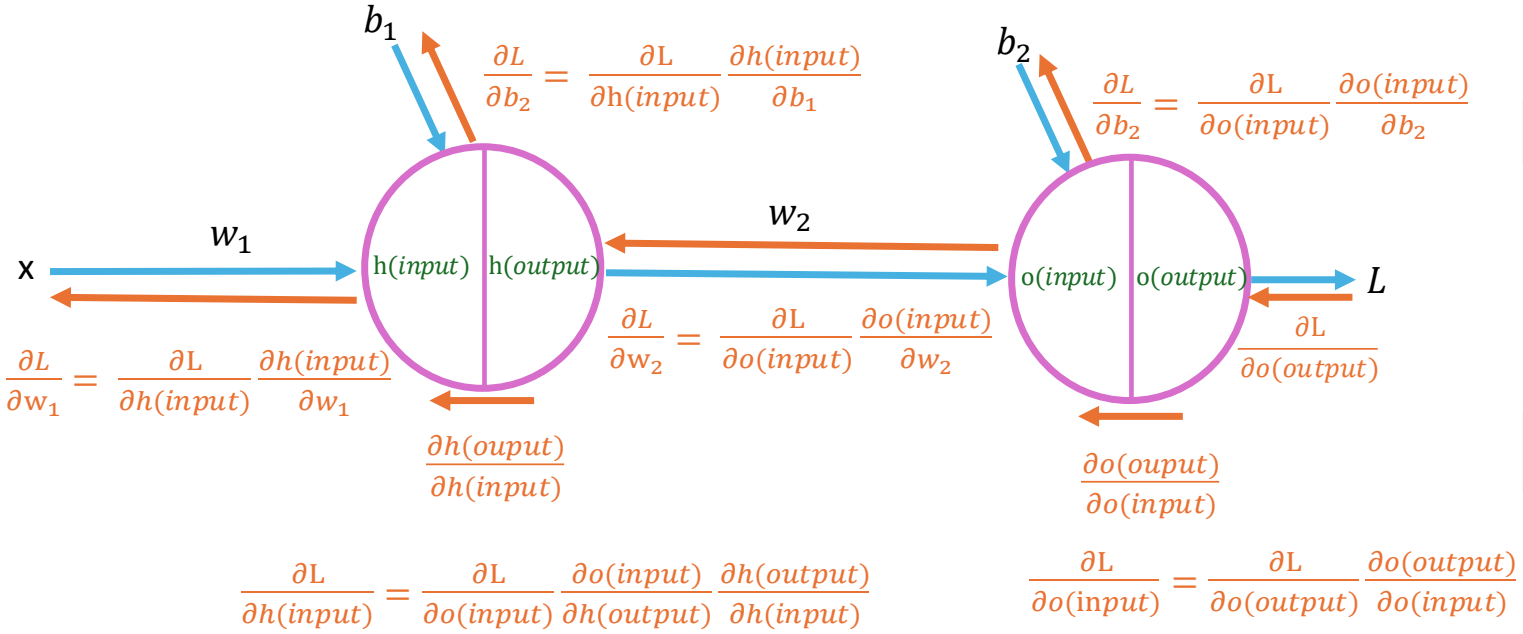
# Example 1

Finally, the gradient of $w_1$ and $b_1$ on total loss will be

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial h(input)} \frac{\partial h(input)}{\partial w_1} = -0.0100 \times x = -0.0100 \times 0.5 = -0.0050$$

and

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial h(input)} \frac{\partial h(input)}{\partial w_1} = -0.0100 \times 1 = -0.0100$$



# Example 1

Step 4: Update parameters

$$w_1^{new} = w_1 - \alpha \frac{\partial L}{\partial w_1} = 0.3 - 0.1 \times (-0.0050) = 0.3005$$

$$b_1^{new} = b_1 - \alpha \frac{\partial L}{\partial b_1} = 0.1 - 0.1 \times (-0.0100) = 0.101$$

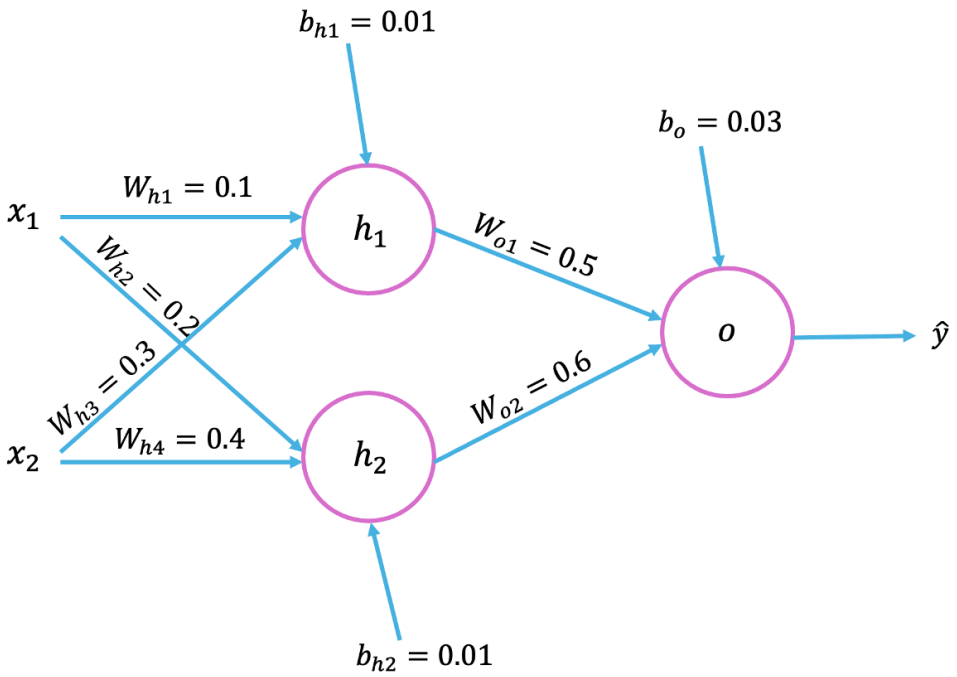
$$w_2^{new} = w_2 - \alpha \frac{\partial L}{\partial w_2} = 0.2 - 0.1 \times (-0.1151) = 0.2115$$

$$b_2^{new} = b_2 - \alpha \frac{\partial L}{\partial b_2} = 0.4 - 0.1 \times (-0.2049) = 0.4205$$

# Example 2

Assume

- Input $x_1 = 0.5$, $x_1 = 0.2$. True outcome $y = 0.7$.
- Initial parameters: $w_{h_1} = 0.1$, $w_{h_2} = 0.2$, $w_{h_3} = 0.3$, $w_{h_4} = 0.4$, $b_{h_1} = 0.01$, $b_{h_2} = 0.02$, $w_{o_1} = 0.5$, $w_{o_2} = 0.6$, $b_o = 0.03$.
- Activation function: sigmoid function. Loss function: $L(y, \hat{y}) = (y - \hat{y})^2$.
- learning rate: 0.1.



# Example 2

```
# import package
import numpy as np

# Activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of activation function
def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)
```

# Example 2

```
# Loss function
def mse_loss(y_true, y_predicted):
    return np.mean(np.square(y_true - y_predicted))

# Derivative of loss function
def mse_loss_derivative(y_true, y_predicted):
    return 2 * (y_predicted - y_true) / y_true.size
```

# Example 2

```
# Input data
X = np.array([[0.5, 0.2]])
# True outcome
y_true = np.array([[0.7]])

# Initial parameters of the hidden layer
W_h = np.array([[0.05, 0.1],
                [0.02, 0.08]])
b_h = np.array([[0.01, 0.02]])

# Initial parameters of the output layer
W_o = np.array([[0.1],
                [0.03]])
b_o = np.array([[0.05]])
```

# Example 2

```
# Step 1: forward pass

# Forward pass: hidden layer
Z_h = np.dot(X, W_h) + b_h
A_h = sigmoid(Z_h)

# Forward pass: output layer
Z_o = np.dot(A_h, W_o) + b_o
A_o = sigmoid(Z_o)
```

# Example 2

```
# Step 2: calculate errors
loss = mse_loss(y_true, A_o)
print("Loss:", loss)

# Step 3: backward pass
# Gradient of the output layer
dLoss_dA_o = mse_loss_derivative(y_true, A_o)
dA_o_dZ_o = sigmoid_derivative(Z_o)
delta_o = dLoss_dA_o * dA_o_dZ_o # Chain rule

# Gradient of the w_o and b_o
grad_W_o = np.dot(A_h.T, delta_o)
grad_b_o = np.sum(delta_o, axis=0, keepdims=True)
```

# Example 2

```
# Gradient of the hidden layer
dLoss_dA_h = np.dot(delta_o, W_o.T)
dA_h_dZ_h = sigmoid_derivative(Z_h)
delta_h = dLoss_dA_h * dA_h_dZ_h # Chain rule

# Gradient of w_h and b_h
grad_W_h = np.dot(X.T, delta_h)
grad_b_h = np.sum(delta_h, axis=0, keepdims=True)
```

# Example 2

```
# Step 4: update parameters
learning_rate = 0.1

W_o -= learning_rate * grad_W_o
b_o -= learning_rate * grad_b_o
W_h -= learning_rate * grad_W_h
b_h -= learning_rate * grad_b_h
```